3 - 监控模块

本节重点

本节我们将学习第三方平台 API 的对接技巧，掌握 HTTP 请求封装的最佳实践，实现知识星球监控功能，并为后续扩展其他平台监控打好基础。

具体内容包括：

- 第三方 API 分析
- 知识星球 API 封装
- 监控逻辑实现
- 监控模块扩展

这一节涉及到真实的第三方 API 调用，建议大家在学习过程中多关注接口文档的阅读技巧和调试方法。

一、第三方 API 分析

为什么选择知识星球?

为什么项目要支持知识星球平台的监控呢?

其实这跟鱼皮本人的需求有关哈哈。

知识星球是一个知识付费和社群运营平台，很多技术大佬都在上面开设了付费圈子。对于圈主来说，及时回复成员的提问是提升服务质量的关键，但人工回复存在几个问题：

1. 时间成本高：需要 7x24 小时关注，消耗大量精力
2. 回复质量不稳定：每个人不可能什么都懂，总有自己的知识边界
3. 重复问题多：很多问题是重复的，但还是需要逐一回复

通过 AI 自动回复，可以很好地解决这些问题。而且知识星球有一个重要特点：**可以限制提问次数**，这样可以防止恶意刷接口消耗 OpenAI 的 token。

知识星球 API 分析

知识星球并没有提供官方的开放 API，所以我们需要通过抓包的方式来分析它的接口。

💡 注意，仅供学习使用，请不要给别人的服务器造成压力！不要获取到不该获取的信息！

1、获取未回答问题列表

通过浏览器开发者工具，我们可以发现获取未回答问题的接口：

请求地址：`https://api.zsxq.com/v2/groups/{groupId}/topics`

请求方法：GET

请求参数：

- `count`：获取数量，建议设置为 20
- `scope`：范围，设置为 `unanswered_questions` 表示未回答的问题

- `groupId`：圈子 ID，从 URL 中获取

请求头：

- `cookie`：用户的登录凭证，需要从浏览器中复制
- `user-agent`：浏览器标识，避免被识别为爬虫

2、回答问题接口

请求地址：`https://api.zsxq.com/v2/topics/{topicId}/answer`

请求方法：POST

请求参数：
▼
```
json
复制代码
{
    "req_data": {
        "text": "回答内容",
        "image_ids": [],
        "silenced": true
    }
}
```

参数说明：

- `text`：回答的文本内容
- `image_ids`：图片 ID 列表，暂时不支持图片回复
- `silenced`：是否静默回复，true 表示不通知提问者

Cookie 获取方法

Cookie 是调用知识星球 API 的关键凭证，获取方法如下：

1. 打开浏览器，登录知识星球
2. 进入你要监控的圈子
3. 按 F12 打开开发者工具
4. 切换到 Network 面板
5. 刷新页面，找到任意一个对 `api.zsxq.com` 的请求
6. 在请求头中找到 `cookie` 字段，复制整个值

注意几个点：

- Cookie 有时效性，过期后需要重新获取
- 不要把 Cookie 提交到公开的代码仓库
- 建议通过环境变量的方式配置 Cookie

二、知识星球 API 封装

选择 HTTP 客户端

Java 生态中有很多 HTTP 客户端可以选择：

- HttpURLConnection：JDK 自带，但使用较为繁琐

- Apache HttpClient：功能强大，但配置复杂
- OkHttp：现代化的 HTTP 客户端，性能优秀
- Hutool Http：基于 HttpURLConnection 封装，使用简单

为了方便，这里我们选择 Hutool 的 HTTP 工具类。Hutool 提供了非常简洁的 API，可以用很少的代码完成复杂的 HTTP 请求。

API 封装

1、请求和响应模型

首先定义请求和响应的数据模型，通过强类型的模型可以避免很多运行时错误。

一般这种代码都可以直接利用 AI 生成了，把接口文档甩给 AI 就好。

获取话题列表请求：

▼
java
复制代码
```
@Data
public class ListTopicsRequest {
    private String groupId;
    private String scope;
    private Integer count;
}
```

获取话题列表响应：

▼
java
复制代码
```
@Data
public class ListTopicsResponse {

    private RespData respData;
    private boolean succeeded;

    @Data
    public static class RespData {
        private List<TopicsItem> topics;
    }

    @Data
    public static class TopicsItem {
        private int readingCount;
        private Question question;
        private boolean answered;
        private String createTime;
        private String type;
        private String topicId;
        private Group group;
        // ... 其他字段
    }

    @Data
    public static class Question {
        private Owner owner;
        private boolean expired;
        private boolean anonymous;
        private String text;
        // ... 其他字段
    }

    // ... 其他内部类
}
```

回答问题请求：

▼

java
复制代码
```
@Data
public class AnswerRequest {

    private String topicId;
    private ReqData req_data;

    @Data
    public static class ReqData {
        private String text;
        private List<String> image_ids;
        private Boolean silenced;
    }
}
```

回答问题响应：

▼

java
复制代码
```
@Data
public class AnswerResponse {

    private RespData respData;
    private boolean succeeded;

    @Data
    public static class Group {
        private long groupId;
        private String name;
        private String type;
    }

    @Data
    public static class Owner {
        private String avatarUrl;
        private long userId;
        private String name;
        private String location;
    }

    @Data
    public static class OwnerDetail {
        private int questionsCount;
        private String joinTime;
    }

    @Data
    public static class Question {
        private Owner owner;
        private boolean expired;
        private Questionee questionee;
        private boolean anonymous;
        private OwnerDetail ownerDetail;
        private String ownerLocation;
        private String text;
    }

    @Data
    public static class Questionee {
        private String avatarUrl;
        private long userId;
        private String name;
        private String alias;
        private String description;
        private String location;
    }

    @Data
    public static class RespData {
        private TopicsItem topics;
    }

    @Data
    public static class TopicsItem {
        private int readingCount;
        private Question question;
        private boolean answered;
        private String createTime;
        private UserSpecific userSpecific;
        private int rewardsCount;
        private String type;
        private boolean digested;
        private int likesCount;
        private int commentsCount;
        private boolean sticky;
        private long topicId;
        private int readersCount;
        private Group group;
    }
```

```
    @Data
    public static class UserSpecific {
        private boolean subscribed;
        private boolean liked;
    }
}
```

2、API 接口实现

核心的思路是构造知识星球 API 的请求，并获取到响应结果。
▼
java
复制代码
```
@Service
public class ZsxqApi {

    private static final String USER_AGENT = "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/111.0";

    /**
     * 获取主题列表
     *
     * @param request
     * @param cookie
     * @return
     */
    public ListTopicsResponse listTopics(ListTopicsRequest request, String cookie) {
        String groupId = request.getGroupId();
        if (StringUtils.isBlank(groupId)) {
            throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 groupId");
        }
        if (StringUtils.isBlank(cookie)) {
            throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 cookie");
        }
        String url = String.format("https://api.zsxq.com/v2/groups/%s/topics", groupId);
        Map<String, Object> stringObjectMap = BeanUtil.beanToMap(request);
        String query = URLUtil.buildQuery(stringObjectMap, StandardCharsets.UTF_8);
        String result = HttpRequest.get(url)
                .header("cookie", cookie)
                .header("user-agent", USER_AGENT)
                .body(query)
                .execute()
                .body();
        return JSONUtil.toBean(result, ListTopicsResponse.class);
    }

    /**
     * 回答问题
     *
     * @param request
     * @param cookie
     * @return
     */
    public AnswerResponse answer(AnswerRequest request, String cookie) {
        String topicId = request.getTopicId();
        if (StringUtils.isBlank(topicId)) {
            throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 topicId");
        }
        if (StringUtils.isBlank(cookie)) {
            throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 cookie");
        }
        String url = String.format("https://api.zsxq.com/v2/topics/%s/answer", topicId);
        String json = JSONUtil.toJsonStr(request);
        String result = HttpRequest.post(url)
                .header("cookie", cookie)
                .header("user-agent", USER_AGENT)
                .body(json)
                .execute()
                .body();
        return JSONUtil.toBean(result, AnswerResponse.class);
    }
}
```

HTTP 封装的最佳实践

再分享一些 HTTP 请求封装的最佳实践，之后对接第三方 API 会更安全。

1、统一的 User-Agent

设置合适的 User-Agent 可以避免被目标网站识别为爬虫，可以直接从自己的浏览器里复制出来（或者让 AI 生成）。
▼
java
复制代码
```
private static final String USER_AGENT =
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/111.0";
```

2、参数校验

在发送请求之前，先校验必要的参数：

▼
java
复制代码
```
if (StringUtils.isBlank(groupId)) {
    throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 groupId");
}
```

3、异常处理

HTTP 请求可能出现各种异常，需要合理处理：

▼
java
复制代码
```
try {
    String result = HttpRequest.get(url)
            .header("cookie", cookie)
            .execute()
            .body();
} catch (Exception e) {
    log.error("请求失败", e);
    throw new BusinessException(ErrorCode.SYSTEM_ERROR, "网络请求失败");
}
```

4、JSON 序列化

使用工具类进行 JSON 的序列化和反序列化，快速获取到响应结果。

▼
java
复制代码
```
// 对象转 JSON
String json = JSONUtil.toJsonStr(request);

// JSON 转对象
ListTopicsResponse response = JSONUtil.toBean(result, ListTopicsResponse.class);
```

此外，还需要注意，有些请求工具是需要手动释放资源的。

三、监控逻辑实现

知识星球监控者

基于前面定义的 Monitor 抽象类，我们来实现知识星球的监控者：

▼
java
复制代码
```
@Slf4j
public class ZsxqMonitor extends Monitor {

    private final ZsxqApi zsxqApi = SpringContextUtils.getBean(ZsxqApi.class);

    private final ZsxqConfig zsxqConfig = SpringContextUtils.getBean(ZsxqConfig.class);

    public ZsxqMonitor(TaskListItem taskListItem) {
        super(taskListItem);
    }

    @Override
    public void onMonitor(Answerer answerer) {
        String taskName = taskListItem.getName();
        log.info("任务 {} 监控开始", taskName);
        String cookie = zsxqConfig.getCookie();
        // 1. 获取未回答的问题列表
        ListTopicsRequest listTopicsRequest = new ListTopicsRequest();
        listTopicsRequest.setCount(20);
        listTopicsRequest.setGroupId(zsxqConfig.getGroupId());
        listTopicsRequest.setScope("unanswered_questions");
        ListTopicsResponse listTopicsResponse = zsxqApi.listTopics(listTopicsRequest, zsxqConfig.getCookie());
        List<ListTopicsResponse.TopicsItem> topics = listTopicsResponse.getRespData().getTopics();
        if (CollectionUtils.isEmpty(topics)) {
            log.info("暂无新提问");
            return;
        }
        for (ListTopicsResponse.TopicsItem topic : topics) {
            String question = topic.getQuestion().getText().trim();
            log.info("{} 收到新提问 \n 问题：{}", taskName, question);
            // 2. 获取回答
            String answer = answerer.doAnswer(question);
```

```
            // 3. 回复
            AnswerRequest answerRequest = new AnswerRequest();
            answerRequest.setTopicId(topic.getTopicId());
            AnswerRequest.ReqData reqData = new AnswerRequest.ReqData();
            reqData.setSilenced(zsxqConfig.getSilenced());
            reqData.setText(answer);
            reqData.setImage_ids(new ArrayList<>());
            answerRequest.setReq_data(reqData);
            AnswerResponse answerResponse = zsxqApi.answer(answerRequest, cookie);
            if (answerResponse.isSucceeded()) {
                log.info("{} 回答成功 \n 问题：{} \n 回答：{}", taskName, question, answer);
            } else {
                log.error("{} 回答失败 \n 问题：{}", taskName, question);
            }
            // 随机缓冲
            try {
                Thread.sleep(1000 + RandomUtil.randomInt(0, 2000));
            } catch (InterruptedException e) {
                throw new BusinessException(ErrorCode.SYSTEM_ERROR, e.getMessage());
            }
        }
        log.info("任务 {} 监控结束", taskName);
    }
}
```

监控逻辑的关键点

上述代码很长，但是我们只需要关注这些关键实现。

1、限流保护

为了避免被平台限流，需要在请求之间添加随机延迟：

▼

java
复制代码

```
// 随机缓冲
try {
    Thread.sleep(1000 + RandomUtil.randomInt(0, 2000));
} catch (InterruptedException e) {
    throw new BusinessException(ErrorCode.SYSTEM_ERROR, e.getMessage());
}
```

2、日志记录

详细的日志记录对于问题排查非常重要：

▼

java
复制代码

```
log.info("任务 {} 监控开始", taskName);
log.info("{} 收到新提问 \n 问题：{}", taskName, question);
log.info("{} 回答成功 \n 问题：{} \n 回答：{}", taskName, question, answer);
log.error("{} 回答失败 \n 问题：{}", taskName, question);
log.info("任务 {} 监控结束", taskName);
```

3、参数校验

在 API 调用前进行必要的参数校验：

▼

java
复制代码

```
if (StringUtils.isBlank(groupId)) {
    throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 groupId");
}
if (StringUtils.isBlank(cookie)) {
    throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 cookie");
}
```

默认监控者实现

我们可以实现一个默认的监控者，便于测试效果：

▼

java
复制代码

```
public class DefaultMonitor extends Monitor {

    public DefaultMonitor(TaskListItem taskListItem) {
        super(taskListItem);
    }
```

```
    @Override
    public void onMonitor(Answerer answerer) {
        String mockMessage = "我是一个新的消息";
        System.out.println(answerer.doAnswer(mockMessage));
    }
}
```

四、监控模块扩展

基于现有的架构，扩展新的监控源非常简单，按照下面的步骤来就好。

1、实现监控者

继承 Monitor 抽象类，实现具体的监控逻辑：

▼
java
复制代码
```
public class NewMonitor extends Monitor {

    public NewMonitor(TaskListItem taskListItem) {
        super(taskListItem);
    }

    @Override
    public void onMonitor(Answerer answerer) {
        // 实现具体的监控逻辑
        // 1. 获取需要处理的消息
        // 2. 调用 answerer.doAnswer() 获取回答
        // 3. 将回答发送到目标平台
    }
}
```

2、更新工厂类

在 MonitorFactory 中添加新的监控者类型：

▼
java
复制代码
```
public static Monitor createMonitor(String monitor, TaskListItem taskListItem) {
    switch (monitor) {
        case "zsxq":
            return new ZsxqMonitor(taskListItem);
        case "new_platform":  // 新增
            return new NewMonitor(taskListItem);
        default:
            return new DefaultMonitor(taskListItem);
    }
}
```

3、配置文件支持

在配置文件中添加新的任务配置：

▼
yaml
复制代码
```
task:
  list:
    - name: new_task
      monitor: new_platform
      answerer: openai
      cron: '0/60 * * * * ?'
```

监控模块的设计亮点

最后，让我们再复习一下监控模块的设计亮点，如果面试官问到，有个印象就好。

1、统一接口

所有监控者都实现同一个接口，保证了一致性：

▼
java
复制代码
```
public abstract void onMonitor(Answerer answerer);
```

2、配置驱动

通过配置文件就可以启用不同的监控源：

▼
yaml

```
复制代码
monitor: zsxq  # 可以改为 wechat、qq 等
```

3、依赖注入

监控者不直接创建回答者，而是通过参数注入：

```
▼
java
复制代码
public void onMonitor(Answerer answerer) {
    String answer = answerer.doAnswer(question);
}
```

4、可扩展性

新增监控源只需要：

- 实现 Monitor 接口
- 更新工厂类
- 添加配置支持

五、扩展思路

1）实现更多平台的监控支持，比如钉钉群、B 站等

2）添加监控规则配置，比如只监控包含特定关键词的问题

3）实现监控数据的统计和分析，比如问题数量、回答成功率等（推荐学习 鱼皮 AI 零代码应用生成平台项目 的第 13 期 —— 可观测性）

4）添加监控状态的健康检查，及时发现和处理异常

5）支持监控结果的通知机制，比如邮件、短信等